

INP Polytech Clermont Ferrand

Rapport du projet ESN12

Laboratoires avancés de co-conception

Effectué par

Mustapha Lahmer
Département de Génie Électrique
Option : Systèmes Informatiques Embarqués

Encadrant :

M. BOUCHAKOUR Mohamed

22/01/2025

Table des matières

1	Introduction	1
2	Conception de l'Architecture	2
2.1	Architecture	2
2.2	Intégration des Composants dans le Platform Designer	3
2.3	Code VHDL	4
2.3.1	Code du 7 segments	4
2.3.2	Code VHDL du projet principal	5
3	Implémentation et Fonctionnalités	6
3.1	Initialisation de l'I2C	6
3.2	Affichage UART	7
3.3	Calibration	7
3.4	Affichage sur 7 Segments	7
3.5	Affichage des axes X, Y et Z avec bouton et interruption	8
3.6	Utilisation du Timer	8
4	Conclusion	9

1 Introduction

Ce projet vise à développer un système permettant d'afficher les données d'accélération d'un gyroscope sur une carte DE10-Lite en temps réel. L'objectif principal est de mesurer les accélérations le long des axes X, Y et Z à intervalles réguliers d'une seconde, puis d'afficher ces valeurs sur l'écran 7 segments de la carte.

Pour atteindre cet objectif, le système repose sur le processeur Nios II, qui permet d'intégrer des fonctionnalités matérielles et logicielles. L'interfaçage avec l'accéléromètre est réalisé à l'aide des protocoles de communication I2C ou SPI, offrant une lecture et une écriture précises des données.

Une étape clé du projet consiste à calibrer les données issues du capteur. Cette calibration, réalisée en exploitant la gravité, garantit une précision optimale des mesures d'accélération, indispensables pour l'affichage fiable des valeurs sur le matériel.

Enfin, pour améliorer l'interactivité, une fonctionnalité utilisateur a été intégrée au système. Un bouton-poussoir permet de basculer facilement entre les valeurs des différents axes, offrant une expérience intuitive et pratique. Ce projet met ainsi en avant l'intégration harmonieuse entre matériel et logiciel dans le cadre d'une solution embarquée.

2 Conception de l'Architecture

2.1 Architecture

L'architecture que j'ai conçue comprend les composants suivants :

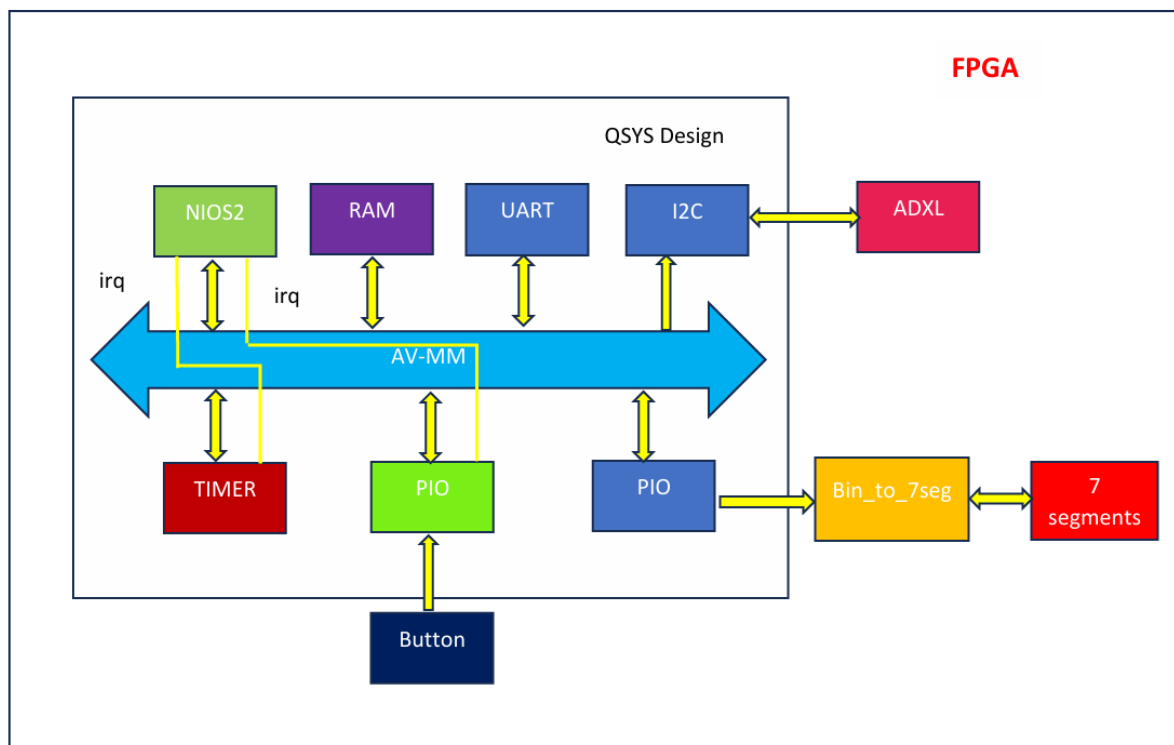


FIGURE 2.1 – Architecture

Le schéma montre l'architecture du système conçu pour lire et afficher les données d'accélération du capteur ADXL sur une carte DE10-Lite. Le processeur Nios II contrôle le système, interfaçant le capteur via I2C pour récupérer les données, qui sont affichées sur un écran 7 segments grâce au module Bin_to_7seg. Un bouton-poussoir permet de basculer entre les axes X, Y et Z, tandis qu'un module Timer assure la synchronisation des lectures à intervalles réguliers (Une seconde). Cette conception modulaire garantit une intégration efficace des composants matériels et logiciels.

2.2 Intégration des Composants dans le Platform Designer

Use	Connections	Name	Description	Export	Clock	Base	End	IRQ
☒		clk_0	Clock Source					
		clk_in	Clock Input	clk	exported			
		clk_in_reset	Reset Input	reset				
		clk	Clock Output	Double-click to export	clk_0			
		clk_reset	Reset Output	Double-click to export				
☒		nios2_gen2_0	Nios II Processor					
		clk	Clock Input	Double-click to export	clk_0			
		reset	Reset Input	Double-click to export	[clk]			
		data_master	Avalon Memory Mapped Master	Double-click to export	[clk]			
		instruction_master	Avalon Memory Mapped Master	Double-click to export	[clk]			
		irq	Interrupt Receiver	Double-click to export	[clk]			IRQ 0
		debug_reset_requ...	Reset Output	Double-click to export	[clk]			IRQ 31
		debug_mem_slave	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0002_0800	0x0002_0fff	
		custom_instructio...	Custom Instruction Master	Double-click to export				
☒		RAM0	On-Chip Memory (RAM or ROM)...					
		clk1	Clock Input	Double-click to export	clk_0			
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk1]	# 0x0001_0000	0x0001_9fff	
		reset1	Reset Input	Double-click to export	[clk1]			
☒		jtag_uart_0	JTAG UART Intel FPGA IP					
		clk	Clock Input	Double-click to export	clk_0			
		reset	Reset Input	Double-click to export	[clk]			
		avalon_jtag_slave	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0002_10b0	0x0002_10b7	
		irq	Interrupt Sender	Double-click to export	[clk]			
☒		opencores_i2c_0	OpenCores I2C					
		clock	Clock Input	Double-click to export	clk_0			
		clock_reset	Reset Input	Double-click to export	[clock]			
		export_0	Conduit	opencores_i2c_0_ex...				
		avalon_slave_0	Avalon Memory Mapped Slave	Double-click to export	[clock]	# 0x0002_1020	0x0002_103f	
		interrupt_sender	Interrupt Sender	Double-click to export	[clock]			
☒		KEY	PIO (Parallel I/O) Intel FPGA IP					
		clk	Clock Input	Double-click to export	clk_0			
		reset	Reset Input	Double-click to export	[clk]			
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0002_10a0	0x0002_10af	
		external_connection	Conduit	key_external_conne...				
		irq	Interrupt Sender	Double-click to export	[clk]			
☒		SEG0	PIO (Parallel I/O) Intel FPGA IP					
		clk	Clock Input	Double-click to export	clk_0			
		reset	Reset Input	Double-click to export	[clk]			
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0002_1090	0x0002_109f	
		external_connection	Conduit	seg0_external_conn...				
☒		SEG1	PIO (Parallel I/O) Intel FPGA IP					
		clk	Clock Input	Double-click to export	clk_0			
		reset	Reset Input	Double-click to export	[clk]			
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0002_1080	0x0002_108f	
		external_connection	Conduit	seg1_external_conn...				
☒		SEG2	PIO (Parallel I/O) Intel FPGA IP					
		clk	Clock Input	Double-click to export	clk_0			
		reset	Reset Input	Double-click to export	[clk]			
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0002_1070	0x0002_107f	
		external_connection	Conduit	seg2_external_conn...				
☒		SEG3	PIO (Parallel I/O) Intel FPGA IP					
		clk	Clock Input	Double-click to export	clk_0			
		reset	Reset Input	Double-click to export	[clk]			
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0002_1060	0x0002_106f	
		external_connection	Conduit	seg3_external_conn...				
☒		SEG4	PIO (Parallel I/O) Intel FPGA IP					
		clk	Clock Input	Double-click to export	clk_0			
		reset	Reset Input	Double-click to export	[clk]			
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0002_1050	0x0002_105f	
		external_connection	Conduit	seg4_external_conn...				
☒		SEG5	PIO (Parallel I/O) Intel FPGA IP					
		clk	Clock Input	Double-click to export	clk_0			
		reset	Reset Input	Double-click to export	[clk]			
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0002_1040	0x0002_104f	
		external_connection	Conduit	seg5_external_conn...				
☒		timer_0	Interval Timer Intel FPGA IP					
		clk	Clock Input	Double-click to export	clk_0			
		reset	Reset Input	Double-click to export	[clk]			
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]	# 0x0002_1000	0x0002_101f	
		irq	Interrupt Sender	Double-click to export	[clk]			

FIGURE 2.2 – Platform Designer

Dans le Platform Designer, j'ai ajouté plusieurs composants essentiels pour la conception du système. Parmi ces composants, on retrouve le processeur Nios II, qui assure le traitement des données, ainsi que le module JTAG UART pour la communication avec l'hôte via UART. Un module OpenCores I2C a été intégré pour gérer la communication I2C avec le capteur ADXL345. J'ai également ajouté des modules PIO pour piloter les différents segments 7-segments (SEG0 à SEG5) et les boutons (KEY). Enfin, un timer a été inclus pour générer des interruptions périodiques, permettant de synchroniser les mises à jour des données affichées et des mesures. Ces composants, interconnectés via le bus Avalon, permettent de concevoir un système complet et fonctionnel pour la gestion des capteurs et des affichages.

2.3 Code VHDL

2.3.1 Code du 7 segments

Le premier code, intitulé `bin_to_7seg`, est dédié à la conversion des valeurs binaires en signaux compatibles avec l'affichage sur 7 segments. Ce composant gère la traduction des chiffres binaires en signaux adaptés aux afficheurs.

```

1  |
2  | library ieee;
3  | use ieee.std_logic_1164.all;
4  | use ieee.numeric_std.all;
5  |
6  | entity bin_to_7seg is
7  |   port(
8  |       input : in std_logic_vector(3 downto 0);
9  |       output : out std_logic_vector(6 downto 0));
10 |
11 | end entity bin_to_7seg;
12 |
13 | architecture rit of bin_to_7seg is
14 |   L
15 |   begin
16 |
17 |       output <= "1000000" when input = "0000" else
18 |                 "1111001" when input = "0001" else
19 |                 "0100100" when input = "0010" else
20 |                 "0110000" when input = "0011" else
21 |                 "0011001" when input = "0100" else
22 |                 "0010010" when input = "0101" else
23 |                 "0000010" when input = "0110" else
24 |                 "1111000" when input = "0111" else
25 |                 "0000000" when input = "1000" else
26 |                 "0010000" when input = "1001";
27 |
28 | end architecture;
```

2.3.2 Code VHDL du projet principal

Ce code VHDL représente le projet principal en intégrant tous les composants essentiels. L'entité project connecte les signaux d'horloge, reset, I2C, boutons (KEY) et afficheurs 7 segments (SEG0 à SEG5). L'architecture rtl utilise le composant principal pour gérer les modules externes et plusieurs instances du composant `bin_to_7seg`.

```

3  library ieee;
4  use ieee.std_logic_1164.all;
5  use ieee.numeric_std.all;
6
7
8  entity project is
9  port(
10     clk : in std_logic := '0';
11     reset_n : in std_logic := '0';
12     scl : inout std_logic := 'X';
13     sda : inout std_logic := 'X';
14     cs_n : out std_logic := '1';
15     key : in std_logic := 'X';
16     segment0 : out std_logic_vector(6 downto 0);
17     segment1 : out std_logic_vector(6 downto 0);
18     segment2 : out std_logic_vector(6 downto 0);
19     segment3 : out std_logic_vector(6 downto 0);
20     segment4 : out std_logic_vector(6 downto 0);
21     segment5 : out std_logic_vector(6 downto 0));
22
23  end entity project;
24
25
26  architecture rtl of project is
27
28     component projet is
29     port (
30         clk_clk : in std_logic := 'X';
31         reset_reset_n : in std_logic := 'X';
32         opencores_i2c_0_export_0_scl_pad_io : inout std_logic := 'X';
33         opencores_i2c_0_export_0_sda_pad_io : inout std_logic := 'X';
34         key_external_connection_export : in std_logic := 'X';
35         seg0_external_connection_export : out std_logic_vector(3 downto 0);
36         seg1_external_connection_export : out std_logic_vector(3 downto 0);
37         seg2_external_connection_export : out std_logic_vector(3 downto 0);
38         seg3_external_connection_export : out std_logic_vector(3 downto 0);
39         seg4_external_connection_export : out std_logic_vector(3 downto 0);
40         seg5_external_connection_export : out std_logic_vector(6 downto 0)
41     );
42     end component projet;
43
44     component bin_to_7seg is
45     port(
46         input : in std_logic_vector(3 downto 0) := (others => 'X');
47         output : out std_logic_vector(6 downto 0));
48     end component bin_to_7seg;
49
50
51     signal tmp0 : std_logic_vector(3 downto 0);
52     signal tmp1 : std_logic_vector(3 downto 0);
53     signal tmp2 : std_logic_vector(3 downto 0);
54     signal tmp3 : std_logic_vector(3 downto 0);
55     signal tmp4 : std_logic_vector(3 downto 0);
56

```

```

57 | begin
58 |   u0 : component projet
59 |   port map (
60 |       clk_clk      => clk,
61 |       reset_reset_n => reset_n,
62 |       opencores_i2c_0_export_0_scl_pad_io => scl,
63 |       opencores_i2c_0_export_0_sda_pad_io => sda,
64 |       key_external_connection_export    => key,
65 |       seg0_external_connection_export    => tmp0,
66 |       seg1_external_connection_export    => tmp1,
67 |       seg2_external_connection_export    => tmp2,
68 |       seg3_external_connection_export    => tmp3,
69 |       seg4_external_connection_export    => tmp4,
70 |       seg5_external_connection_export    => segment5
71 |   );
72 |
73 |
74 |
75 |   convert0 : component bin_to_7seg
76 |   port map (
77 |       input => tmp0,
78 |       output => segment0);
79 |
80 |   convert1 : component bin_to_7seg
81 |   port map (
82 |       input => tmp1,
83 |       output => segment1);
84 |
85 |   convert2 : component bin_to_7seg
86 |   port map (
87 |       input => tmp2,
88 |       output => segment2);
89 |
90 |   convert3 : component bin_to_7seg
91 |   port map (
92 |       input => tmp3,
93 |       output => segment3);
94 |
95 |   convert4 : component bin_to_7seg
96 |   port map (
97 |       input => tmp4,
98 |       output => segment4);
99 |
100 | end architecture;
101 |

```

FIGURE 2.3 – projet principal

3 Implémentation et Fonctionnalités

3.1 Initialisation de l'I2C

Pour établir une communication bidirectionnelle avec le capteur ADXL345, j'ai initialisé l'interface I2C. La fonction **I2C_init** configure le périphérique I2C avec une fréquence

d'horloge de 100 kHz, assurant une communication stable et adaptée aux spécifications du capteur. Ensuite, j'ai utilisé une commande de démarrage **I2C_start** pour vérifier que le capteur, à l'adresse 0x1d, était correctement connecté au bus. Cette configuration garantit que les opérations de lecture et d'écriture peuvent être effectuées sans erreur, posant ainsi une base solide pour la gestion des données du capteur.

3.2 Affichage UART

Pour surveiller directement les données du capteur ADXL345, j'ai utilisé l'affichage UART. J'ai commencé par lire les registres des axes X, Y et Z : X0 (0x32), X1 (0x33), Y0 (0x34), Y1 (0x35), Z0 (0x36) et Z1 (0x37) via le protocole I2C. Les bits de poids faible (LSB) et de poids fort (MSB) ont été combinés en effectuant un décalage suivi d'une opération OU bit à bit, afin d'obtenir des valeurs brutes de 16 bits pour chaque axe.

Ces données ont ensuite été affichées en hexadécimal grâce à la fonction **alt_printf**. Ce processus m'a permis de visualiser les données en temps réel, facilitant ainsi le débogage et l'analyse des mesures. De plus, l'affichage UART a été un outil pratique pour vérifier que le capteur fonctionnait correctement avant de passer à des affichages plus complexes, comme celui sur 7 segments.

3.3 Calibration

Pour garantir la précision des mesures du capteur ADXL345, j'ai effectué une calibration en ajustant les registres d'offset pour les axes X, Y et Z. J'ai commencé par définir les registres OffsetX (0x1E), OffsetY (0x1F) et OffsetZ (0x20) à des valeurs initiales, supprimant ainsi toute correction d'offset.

Ensuite, j'ai analysé les mesures obtenues sur l'UART. Par exemple, pour l'axe Z, j'ai observé une valeur brute qui, après mise à l'échelle, ne correspondait pas à la valeur attendue pour un état à plat. J'ai progressivement ajusté le registre OffsetZ jusqu'à atteindre la valeur correcte.

La même méthode a été appliquée aux axes X et Y pour obtenir une valeur correspondant à leur état d'équilibre. Après chaque modification, une temporisation a été insérée pour permettre au capteur de stabiliser ses registres. Cette procédure a permis d'améliorer la fiabilité et la précision des données.

3.4 Affichage sur 7 Segments

Pour afficher les valeurs des axes de manière lisible et intuitive, j'ai implémenté un affichage sur 7 segments. J'ai commencé par effectuer des tests sur un seul axe, en convertissant les données brutes du capteur en valeurs signées. Ces valeurs ont été mises à l'échelle à l'aide d'un facteur de 3,9 pour obtenir une unité en milligravités (mg), puis décomposées en chiffres individuels pour être affichées. Une fois le fonctionnement validé pour cet axe, j'ai appliqué le même processus aux deux autres axes.

Pour gérer efficacement l’affichage sur 7 segments, j’ai utilisé un code VHDL nommé `bin_to_7seg`, qui assure la conversion des valeurs binaires en un format compatible avec les afficheurs. De plus, j’ai réservé un segment spécifique pour afficher un signe négatif lorsque la valeur est inférieure à zéro, ce qui permet de distinguer clairement les mesures négatives des positives. Les chiffres et le signe ont été transmis aux afficheurs via les registres correspondants.

3.5 Affichage des axes X, Y et Z avec bouton et interruption

J’ai conçu une méthode interactive pour basculer entre l’affichage des axes X, Y et Z en utilisant une interruption déclenchée par un bouton. Chaque appui sur le bouton appelle une routine d’interruption qui modifie dynamiquement les registres associés à l’axe affiché. Grâce à cette approche, je peux visualiser les valeurs de chaque axe à tour de rôle.

3.6 Utilisation du Timer

J’ai configuré un timer matériel pour générer des interruptions toutes les secondes, garantissant une mise à jour automatique et régulière des données affichées. À chaque interruption, j’ai lu les mesures des axes via I2C, affiché les données sur l’UART et mis à jour les afficheurs 7 segments. Ce délai d’une seconde a été choisi pour offrir un rythme d’actualisation adapté aux besoins de visualisation tout en limitant la sollicitation du processeur.

4 Conclusion

Ce projet a abouti à la conception et à l'implémentation d'un système embarqué capable de mesurer et d'afficher en temps réel les données d'accélération provenant du capteur ADXL345. En combinant des technologies matérielles, comme le processeur Nios II et le bus Avalon, avec des solutions logicielles telles que la gestion des communications I2C, la calibration et l'affichage sur 7 segments, l'objectif principal a été atteint avec succès. Chaque composant, du module timer à la gestion des interruptions via un bouton-poussoir, a contribué à garantir la fiabilité, la précision et l'interactivité du système.

Le projet met en avant l'importance d'une approche modulaire et intégrée dans la conception de systèmes embarqués. Le développement des codes VHDL dédiés et l'utilisation des outils du Platform Designer ont permis de relever les défis techniques tout en renforçant mes compétences en co-conception matérielle et logicielle.